DATA STRUCTURES – SET 1

Section 1: Arrays & Searching

1. Which of the following is the time complexity of accessing an element in an array by index?

   a) O(n)

   b) O(log n)

   c) O(1)

   d) O(n log n)

   Answer: c) O(1)

   Explanation: Arrays allow constant-time access to any element using its index due to contiguous memory allocation.

2. What is a disadvantage of static arrays?

   a) Cannot access randomly

   b) They use excess memory if underutilized

   c) Takes less space than dynamic arrays

   d) Very slow to access

   Answer: b) They use excess memory if underutilized

   Explanation: Fixed-size arrays may waste space if only partially used due to unused slots.

3. What does this code implement?

```
int max = arr[0];
for (int i = 1; i < arr.length; i++)
  if (arr[i] > max) max = arr[i];
```

   a) Bubble sort

   b) Linear search

   c) Binary search

   d) Selection sort

   Answer: b) Linear search

   Explanation: Scans each element sequentially to find max.

4. Java snippet: what's printed?

```
int[] a = {10, 20, 30, 40};
System.out.println(a[a.length - 2]);
```

a) 30

b) 40

c) 20

d) Error

Answer: a) 30

Explanation: Index length-2 = 2, so a[2] = 30.

5. Which sorting algorithm is best for small, almost sorted arrays?

a) Bubble Sort

b) Insertion Sort

c) Merge Sort

d) Quick Sort

Answer: b) Insertion Sort

Explanation: Insertion Sort performs very efficiently (near linear time) on small or nearly sorted data.

6. Binary search requires which condition?

a) Duplicate elements

b) Hash function

c) Data sorted in advance

d) Recursion only

Answer: c) Data sorted in advance

Explanation: Binary search divides the search space in half repeatedly; requires sorted data.

7. What's the output of this C code?

```
int arr[] = {2, 4, 6, 8};
printf("%d", *(arr + 2));
```

a) 2

b) 4

c) 6

d) 8

Answer: c) 6

Explanation: Pointer arithmetic: *(arr + 2) accesses arr[2] = 6.

Section 2: Stacks & Queues

8. Which structure is used to check balanced parentheses?

a) Queue

b) Array

c) Stack

d) Heap

Answer: c) Stack

Explanation: Opening parentheses are pushed; matching closes pop—empty at end indicates balance.

9. Which DS is used in C for queue?

```
int front=-1, rear=-1;
int queue[100];
```

a) Stack

b) Linked list

c) Array

d) Hash map

Answer: c) Array

Explanation: Static queues often use arrays with front, rear.

10. Which queue op is invalid?

```
int dequeue() {
  return queue[front++];
}
```

a) push

b) pop

c) enqueue

d) dequeue

Answer: b) pop

Explanation: pop is stack-specific, not queue.

11. Java stack snippet prints?

```
Stack<Integer> s = new Stack<>();
s.push(1); s.push(2); s.pop(); s.push(3);
System.out.println(s.peek());
```

a) 1

b) 2

c) 3

d) Error

Answer: c) 3

Explanation: Push 1, push 2, pop (removes 2), push 3 → top = 3.

12. Stacks are essential for implementing recursion because they:

a) Are easy to implement

b) Maintain return addresses and local variables

c) Take less memory

d) Provide faster execution

Answer: b) Maintain return addresses and local variables

Explanation: Each recursive call's state is held via a call stack.

13. Why use a circular queue instead of a linear one?

a) Uses more memory

b) To reuse freed space after dequeues

c) Easier to implement

d) Has no front and rear

Answer: b) To reuse freed space after dequeues

Explanation: Circular queues wrap around, preventing buffer underutilization.

🔹 Section 3: Linked Lists

14. Compared to arrays, linked lists allow:

a) Random access in O(1) time

b) Constant-time insertion/deletion anywhere

c) Better memory allocation

d) Sorting in less time

Answer: b) Constant-time insertion/deletion anywhere

Explanation: With a pointer to node, linked lists enable O(1) insertions/deletions.

15. Deleting a given middle node (pointer provided) in a singly linked list takes:

a) O(1)

b) O(n)

c) O(log n)

d) O(n^2)

Answer: a) O(1)

Explanation: Copy subsequent node data and bypass it; avoids traversal.

16. Java singly linked list snippet prints?

```
class Node { int data; Node next; }
Node head = new Node(); head.data = 1;
head.next = new Node(); head.next.data = 2;
System.out.println(head.next.data);
```

a) 1

b) 2

c) 0

d) Null

Answer: b) 2

Explanation: Next node's data is set to 2.

17. To reverse a singly linked list in-place, the time complexity is:

a) O(1)

b) O(log n)

c) O(n)

d) O(n log n)

Answer: c) O(n)

Explanation: One full traversal with link reversal.

18. Which is true about deletion in singly linked list (C)?

```
if (temp->next != NULL)
  temp->next = temp->next->next;
```

a) Deletes tail

b) Deletes head

c) Deletes a middle node

d) Deletes last two nodes

Answer: c) Deletes a middle node

Explanation: Adjusts next pointer to skip over next node.

19. Which scenario causes memory leak in linked list?

```
Node* temp = head;

head = head->next;

// missing free(temp);
```

a) Inserting nodes

b) Skipping head

c) Deleting without free

d) Using malloc

Answer: c) Deleting without free

Explanation: Memory isn't reclaimed without free()

20. What is big-O time for inserting at the end of a singly linked list when tail pointer exists?

a) O(1)

b) O(log n)

c) O(n)

d) O(n log n)

Answer: a) O(1)

Explanation: With tail pointer, insertion at end is constant time.

Section 4: Trees

21. An AVL tree guarantees:

a) Faster search than binary search tree

b) Height difference of subtrees ≤ 1

c) O(n) insertion time

d) More nodes than BST

Answer: b) Height difference of subtrees ≤ 1

Explanation: AVL is a self-balancing BST with height balance rule.

22. What happens if binary search tree receives sorted input?

a) Balanced BST

b) Height = O(log n)

c) Degraded to linked list

d) Random behavior

Answer: c) Degraded to linked list

Explanation: All nodes go to right, making it linear.

23. **What does this code do?**

```
int t1(Node node) {
  if (node == null) return 0;
  return 1 + Math.max(t1(node.left), t1(node.right));
}
```

a) Check balance

b) Count nodes

c) Get tree height

d) Preorder traversal

Answer: c) Get tree height

Explanation: Returns max depth from root to leaf

24. Which data structure is used for range searching?

a) Stack

b) K-d tree

c) Hash Table

d) Heap

Answer: b) K-d tree

Explanation: K-d trees partition space for efficient multidimensional range queries.

25. What is printed?

```
void mirror(Node node) {
   if (node == null) return;
   mirror(node.left);
   mirror(node.right);
   // Swap children
   Node temp = node.left;
   node.left = node.right;
   node.right = temp;
}
Node root = new Node(1);
root.left = new Node(2);
root.right = new Node(3);
mirror(root);
System.out.println(root.left.data + " " + root.right.data);
```

a) 2 3

b) 3 2

c) 1 2

d) 3 1

Answer: b) 3 2

Explanation: mirror() swaps left and right children recursively.

Section 5: Graphs

26. Which DS is used for BFS traversal?

a) Stack

b) Queue

c) Tree

d) Linked List

Answer: b) Queue

Explanation: BFS explores neighbors level by level using FIFO behavior.

27. What does this return?

```
boolean hasCycle(int v, boolean[] visited, boolean[] recStack,
List<List<Integer>> adj) {
   if (recStack[v]) return true;
   if (visited[v]) return false;

   visited[v] = true;
   recStack[v] = true;

   for (int u : adj.get(v)) {
      if (hasCycle(u, visited, recStack, adj)) return true;
   }

   recStack[v] = false;
   return false;
}
```

If the graph is:

0 → 1 → 2 → 0

a) No cycle

b) Compilation Error

c) Cycle exists

d) Infinite loop

Answer: c) Cycle exists

Explanation: Cycle 0→1→2→0 → recStack[v] catches it during back-edge visit.

28. What is the time complexity of BFS with adjacency list and V = vertices, E = edges?

```
for (int i = 0; i < V; i++) {
   for (int neighbor : adj.get(i)) {
      // visit each edge
   }
}
```

a) $O(V^2)$

b) O(E log V)

c) O(V + E)

d) $O(E^2)$

Answer: c) O(V + E)

Explanation: Every vertex and every edge is visited once in BFS.

29. Which traversal is best for topological sort in a DAG?

```
void topoSort(int v, boolean[] visited, Stack<Integer> stack) {
   visited[v] = true;
   for (int u : adj.get(v)) {
      if (!visited[u]) topoSort(u, visited, stack);
   }
   stack.push(v);
}
```

a) BFS

b) DFS

c) Inorder

d) Postorder

Answer: b) DFS

Explanation: DFS helps in ordering vertices post-child processing (topo sort logic).

Section 6: Hashing

30. With linear probing, if collision occurs the algorithm:

a) Deletes key

b) Probes next slot until empty

c) Sorts array

d) Creates new hash

Answer: b) Probes next slot until empty

Explanation: Linear probing linearly searches for next free slot.

31. Hash tables are preferred for:

a) Sorting large files

b) Fast average-time lookups

c) Storing ordered data

d) Insertion sort

Answer: b) Fast average-time lookups

Explanation: Hash tables provide O(1) average lookup time.

32. What will this code output

```
String str = "banana";

Map<Character, Integer> map = new HashMap<>();

for (char ch : str.toCharArray()) {

   map.put(ch, map.getOrDefault(ch, 0) + 1);

}
```

```
System.out.println(map.get('a'));
```

a) 2

b) 3

c) 1

d) NullPointerException

Answer: b) 3

Explanation: 'a' appears 3 times. getOrDefault(ch, 0) initializes to 0 if absent.

33. What will this code print?

```
Map<Integer, String> map = new LinkedHashMap<>();

map.put(2, "B");

map.put(1, "A");

map.put(3, "C");

for (String value : map.values()) {

   System.out.print(value + " ");

}
```

a) A B C

b) B A C

c) C A B

d) B C A

Answer: b) B A C

Explanation: LinkedHashMap maintains insertion order.

- **Section 7: Recursion & Complexity**

34. What is the output of the following recursive function?

```
int mystery(int n) {
   if (n == 0) return 0;
   return n + mystery(n - 1);
}
System.out.println(mystery(4));
```

a) 10
b) 4
c) 6
d) 0

Answer: a) 10
Explanation: It computes sum of first n natural numbers: 4 + 3 + 2 + 1 + 0 = 10.

35. What is the time complexity of the below recursive function?

```
int foo(int n) {
   if (n <= 1) return 1;
   return foo(n - 1) + foo(n - 1);
}
```

a) O(n)
b) O(n^2)
c) O(2^n)
d) O(log n)

Answer: c) O(2^n)
Explanation: Two recursive calls each time, like a binary tree → exponential growth.

36. Identify the problem in this recursive C code:

```
int sum(int n) {

   return n + sum(n - 1);

}
```

a) Returns correct result

b) May cause stack overflow

c) No base case leads to infinite recursion

d) Prints wrong result

Answer: c) No base case leads to infinite recursion

Explanation: Missing base case like if (n <= 0) return 0.

Section 8: Heaps & Priority Queues

37. A ternary heap is:

a) Heap using 2 trees

b) A priority queue where each node has 3 children

c) Hash heap

d) Sorted array

Answer: b) A priority queue where each node has 3 children

Explanation: Ternary heaps generalize binary heaps to three children.

38. Heap is best used to:

a) Sort via min/max efficiently

b) Random access

c) Tree balancing

d) Hash collisions

Answer: a) Sort via min/max efficiently

Explanation: Extracting min/max yields heap sort in O(n log n).

39. What is the output of this Java snippet?

```
PriorityQueue<Integer> maxHeap = new
PriorityQueue<>(Collections.reverseOrder());

maxHeap.add(5);
```

```
maxHeap.add(2);

maxHeap.add(8);

System.out.println(maxHeap.poll());
```

a) 2

b) 5

c) 8

d) 0

Answer: c) 8

Explanation: Reverse order creates a **max-heap**; poll() returns highest value (8).

40. What does the following function do?

```
void heapify(int arr[], int n, int i) {
   int smallest = i;
   int l = 2 * i + 1;
   int r = 2 * i + 2;
   if (l < n && arr[l] < arr[smallest]) smallest = l;
   if (r < n && arr[r] < arr[smallest]) smallest = r;
   if (smallest != i) {
      int temp = arr[i]; arr[i] = arr[smallest]; arr[smallest] = temp;
      heapify(arr, n, smallest);
   }
}
```

a) Builds a max heap

b) Performs heap sort

c) Restores min-heap property at index i

d) Deletes an element

Answer: c) Restores min-heap property at index i

Explanation: Checks left and right children, swaps if needed recursively.

🔹 Section 9: Strings & Miscellaneous

41. To check if a word is palindrome, easiest DS is:

a) Queue

b) Stack

c) Array

d) Graph

Answer: b) Stack

Explanation: Push letters and pop for reverse sequence comparison.

42. What does this C code print?

```
char str1[] = "Placement";

char str2[] = "Test";

strcpy(str1 + 4, str2);

printf("%s", str1);
```

a) PlacTest

b) PlacPlacement

c) Test

d) PlaTest

Answer: a) PlacTest

Explanation: strcpy(str1 + 4, str2) overwrites from index 4 onward in str1.

43. Which DS combines a queue and stack's properties (insert/delete at both ends)?

a) Tree

b) Deque

c) Circular queue

d) Hash map

Answer: b) Deque

Explanation: Allows operations at both front and end.

44. Which method is best for comparing two strings for equality in C/C++?

a) == operator

b) strcmp() function

c) equals() method

d) strcat() function

Answer: b) strcmp() function

Explanation: In C/C++, strcmp() compares two strings character-by-character; == compares addresses.

🔹 Section 10: Advanced

45. Disjoint set is best for:

a) Binary heaps

b) Managing disjoint network sets

c) Sorting

d) Hash collisions

Answer: b) Managing disjoint network sets

Explanation: DSU structure supports union-find operations.

46. Suffix tree is used for:

a) Binary search

b) String pattern search in O(m)

c) Stack balancing

d) Recursion trace

Answer: b) String pattern search in O(m)

Explanation: Suffix trees allow fast substring queries.

47. Which data structure helps implement backtracking algorithms?

a) Queue

b) Heap

c) Stack

d) Graph

Answer: c) Stack

Explanation: Backtracking requires returning to previous states—stack is ideal.

48. Which operation is fastest in a hash table?

a) Sorting

b) Searching

c) Traversal

d) Deletion

Answer: b) Searching

Explanation: Average-case search time in hash tables is O(1).

49. Which DS is used in Dijkstra's algorithm for shortest path?

a) Hash table

b) Queue

c) Priority queue (min-heap)

d) Stack

Answer: c) Priority queue (min-heap)

Explanation: Min-heap ensures extraction of minimum-distance node efficiently.

50. What is the purpose of the sentinel node in linked lists?

a) To sort faster

b) To simplify boundary conditions

c) To hold max value

d) To store next pointer only

Answer: b) To simplify boundary conditions

Explanation: Sentinel node avoids null checks and simplifies edge cases.